



정적 분석 골탕먹이기 논문 리뷰

정원준

2026년 3월 6일

ROPAS@SNU

목차

논문 소개

배경

목표

주요 아이디어

세 가지 방법

결론

한계점 및 발전 방향

논문 소개

무슨 논문인가

무슨 논문인가

- 정적 분석의 취약점을 파고드는 논문

무슨 논문인가

- 정적 분석의 취약점을 파고드는 논문
- 연구실, SIGPL, 신년교례회

무슨 논문인가

- 정적 분석의 취약점을 파고드는 논문
- 연구실, SIGPL, 신년교례회
- 정적 분석기가 실제보다 크게 어림잡는 프로그램을 만드는 방법을 제시

무슨 논문인가

- 정적 분석의 취약점을 파고드는 논문
- 연구실, SIGPL, 신년교례회
- 정적 분석기가 실제보다 크게 어림잡는 프로그램을 만드는 방법을 제시
- 그러나, 그 구체적인 알고리즘까지는 찾지 못함

제목

Extensionality:

$$P_1 \approx P_2 \stackrel{\text{def}}{\iff} \llbracket P_1 \rrbracket = \llbracket P_2 \rrbracket$$

제목

Abstract Extensionality:

$$P_1 \approx P_2 \stackrel{\text{def}}{\iff} \llbracket P_1 \rrbracket = \llbracket P_2 \rrbracket$$

$$P_1 \approx^{\mathcal{A}} P_2 \stackrel{\text{def}}{\iff} \llbracket P_1 \rrbracket^{\mathcal{A}} = \llbracket P_2 \rrbracket^{\mathcal{A}}$$

제목

Abstract Extensionality: On the Properties of Incomplete Abstract Interpretations

$$\begin{aligned} P_1 \approx P_2 &\stackrel{\text{def}}{\iff} \llbracket P_1 \rrbracket = \llbracket P_2 \rrbracket \\ P_1 \approx^{\mathcal{A}} P_2 &\stackrel{\text{def}}{\iff} \llbracket P_1 \rrbracket^{\mathcal{A}} = \llbracket P_2 \rrbracket^{\mathcal{A}} \end{aligned}$$

배경

의미구조

$$\llbracket \text{skip} \rrbracket S \stackrel{\text{def}}{=} S$$

$$\llbracket x := a \rrbracket S \stackrel{\text{def}}{=} \{ \mathfrak{m} [x \mapsto \langle a \rangle \mathfrak{m}] \mid \mathfrak{m} \in S \}$$

$$\llbracket P_1; P_2 \rrbracket S \stackrel{\text{def}}{=} \llbracket P_2 \rrbracket (\llbracket P_1 \rrbracket S)$$

$$\llbracket \text{if } b \text{ then } P_1 \text{ else } P_2 \rrbracket S \stackrel{\text{def}}{=} \llbracket P_1 \rrbracket (\llbracket b \rrbracket S) \cup \llbracket P_2 \rrbracket (\llbracket \neg b \rrbracket S)$$

$$\llbracket \text{while } b \text{ do } P \rrbracket S \stackrel{\text{def}}{=} \llbracket \neg b \rrbracket (\text{lfp}(\lambda T. S \cup \llbracket P \rrbracket (\llbracket b \rrbracket T))).$$

요약의미구조

$$\llbracket \text{skip} \rrbracket^{\mathcal{A}} S^{\#} \stackrel{\text{def}}{=} S^{\#}$$

$$\llbracket x := a \rrbracket^{\mathcal{A}} S^{\#} \stackrel{\text{def}}{=} \alpha(\{ \mathfrak{m}[x \mapsto \langle a \rangle \mathfrak{m}] \mid \mathfrak{m} \in \gamma(S^{\#}) \})$$

$$\llbracket P_1; P_2 \rrbracket^{\mathcal{A}} S^{\#} \stackrel{\text{def}}{=} \llbracket P_2 \rrbracket^{\mathcal{A}} (\llbracket P_1 \rrbracket^{\mathcal{A}} S^{\#})$$

$$\llbracket \text{if } b \text{ then } P_1 \text{ else } P_2 \rrbracket^{\mathcal{A}} S^{\#} \stackrel{\text{def}}{=} \llbracket P_1 \rrbracket^{\mathcal{A}} (\llbracket b \rrbracket^{\mathcal{A}} S^{\#}) \sqcup \llbracket P_2 \rrbracket^{\mathcal{A}} (\llbracket \neg b \rrbracket^{\mathcal{A}} S^{\#})$$

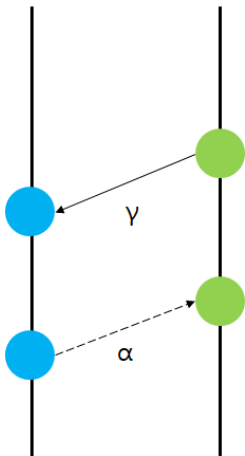
$$\llbracket \text{while } b \text{ do } P \rrbracket^{\mathcal{A}} S^{\#} \stackrel{\text{def}}{=} \llbracket \neg b \rrbracket^{\mathcal{A}} (lfp(\lambda T^{\#}. S^{\#} \sqcup \llbracket P \rrbracket^{\mathcal{A}} \llbracket b \rrbracket^{\mathcal{A}} T^{\#}))$$

가정

1. Galois insertion

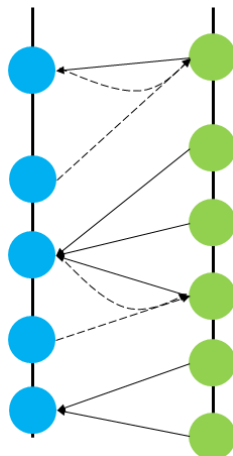
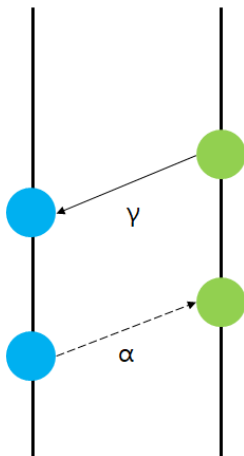
가정

1. Galois insertion



가정

1. Galois insertion

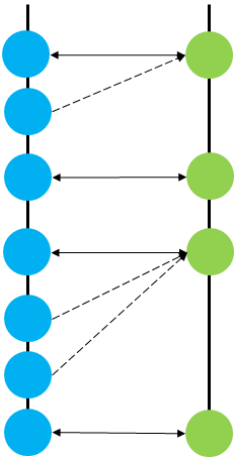


가정

1. Galois insertion

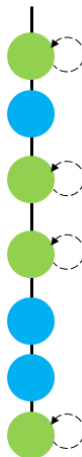
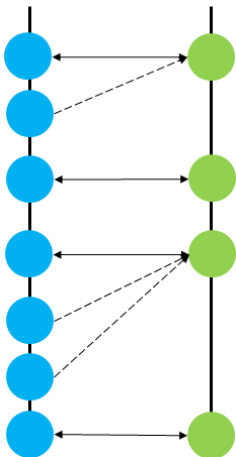
가정

1. Galois insertion



가정

1. Galois insertion



가정

1. Galois insertion

가정

1. Galois insertion
2. strict or trivial domain

가정

1. Galois insertion
2. strict or trivial domain
3. recursive or trivial abstraction

가정

1. Galois insertion
2. strict or trivial domain
3. recursive or trivial abstraction

의미 있고, 깔끔한 분석기를 가정

가정

1. Galois insertion
2. strict or trivial domain
3. recursive or trivial abstraction

의미 있고, 깔끔한 분석기를 가정

4. $x := e$ 를 최고로 잘 어림잡는 분석기를 가정

가정

1. Galois insertion
2. strict or trivial domain
3. recursive or trivial abstraction

의미 있고, 깔끔한 분석기를 가정

4. $x := e$ 를 최고로 잘 어림잡는 분석기를 가정

→ 약한 분석기에 대해서는 당연히 공격 가능

목표

정적 분석기 골탕먹이기

입력 프로그램 P 가 주어졌을 때, P 의 실행 의미는 보존하면서 요약해석 결과는 더 커지게 하는 변환 $\tau(P)$ 를 만들자

정적 분석기 골탕먹이기

입력 프로그램 P 가 주어졌을 때, P 의 실행 의미는 보존하면서 요약해석 결과는 더 커지게 하는 변환 $\tau(P)$ 를 만들자

- (1) *Semantics preservation*: For each $P \in \text{Imp}$: $\llbracket P \rrbracket = \llbracket \tau(P) \rrbracket$.
- (2) *Incompleteness*: There exists $S \in \widehat{\wp}(\mathbb{S})$: $\alpha(\llbracket \tau(P) \rrbracket S) < \llbracket \tau(P) \rrbracket^{\mathcal{A}} \alpha(S)$.

연구 주제와 비교

non-relational domain 가정. 실행 의미는 τ 이 아니지만, 요약해석 결과는 τ 이 되는 프로그램 P 를 만들자(특정 변수 x 에 대해서)

연구 주제와 비교

non-relational domain 가정. 실행 의미는 $\top$ 이 아니지만, 요약해석 결과는 $\top$ 이 되는 프로그램 P 를 만들자(특정 변수 x 에 대해서)

$$(\alpha(\llbracket P \rrbracket \$))_x \neq \top$$

$$(\llbracket P \rrbracket^{\mathcal{A}} \alpha(\$))_x = \top$$

연구 주제와 비교

non-relational domain 가정. 실행 의미는 $\top$ 이 아니지만, 요약해석 결과는 $\top$ 이 되는 프로그램 P 를 만들자(특정 변수 x 에 대해서)

$$(\alpha(\llbracket P \rrbracket \$))_x \neq \top$$

$$(\llbracket P \rrbracket^{\mathcal{A}} \alpha(\$))_x = \top$$

$$\alpha(\llbracket P \rrbracket \$) < \llbracket P \rrbracket^{\mathcal{A}} \alpha(\$)$$

연구 주제와 비교

non-relational domain 가정. 실행 의미는 $\top$ 이 아니지만, 요약해석 결과는 $\top$ 이 되는 프로그램 P 를 만들자(특정 변수 x 에 대해서)

$$(\alpha(\llbracket P \rrbracket S))_x \neq \top$$

$$(\llbracket P \rrbracket^{\mathcal{A}} \alpha(S))_x = \top$$

$$\alpha(\llbracket P \rrbracket S) < \llbracket P \rrbracket^{\mathcal{A}} \alpha(S)$$

- (1) *Semantics preservation*: For each $P \in \text{Imp}$: $\llbracket P \rrbracket = \llbracket \tau(P) \rrbracket$.
- (2) *Incompleteness*: There exists $S \in \widehat{\wp}(S) : \alpha(\llbracket \tau(P) \rrbracket S) < \llbracket \tau(P) \rrbracket^{\mathcal{A}} \alpha(S)$.

주요 아이디어

주요 아이디어

요약해석이 제대로 걸러내지 못하는 상태 집합(if문)을 만들자

주요 아이디어

요약해석이 제대로 걸러내지 못하는 상태 집합(if문)을 만들자

- 1 if 절대 통과 못하는 조건문
- 2 이상한 일을 한다
- 3 P

주요 아이디어

요약해석이 제대로 걸러내지 못하는 상태 집합(if문)을 만들자

- 1 if 절대 통과 못하는 조건문
- 2 이상한 일을 한다
- 3 P

요약해석이 제대로 요약하지 못하는 정수 집합을 만들자

주요 아이디어

요약해석이 제대로 걸러내지 못하는 상태 집합(if문)을 만들자

- 1 if 절대 통과 못하는 조건문
- 2 이상한 일을 한다
- 3 P

요약해석이 제대로 요약하지 못하는 정수 집합을 만들자

$$1 \quad x = 1 \text{ or } 5$$

세 가지 방법

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$P = x := x + 1$$

$$\text{In}(S) = (x = 1) \vee (x = 3)$$

$$\tau(P) \stackrel{\text{def}}{=} \begin{array}{l} \mathbf{if} \text{ In}(S) \mathbf{ then} \\ \quad \mathbf{if} \neg \text{In}(S) \mathbf{ then} (y := 2; x := 0) \\ \quad \mathbf{else} x := x + 1 \text{ (@1)} \\ \mathbf{else} x := x + 1 \text{ (@2)} \end{array}$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$P = x := x + 1$$

$$\text{In}(S) = (x = 1) \vee (x = 3)$$

$\tau(P) \stackrel{\text{def}}{=} \text{if } \text{In}(S) \text{ then}$
 if $\neg \text{In}(S)$ **then** $(y := 2; x := 0)$
 else $x := x + 1$ (@1)
 else $x := x + 1$ (@2)

$$\llbracket \tau(P) \rrbracket = \llbracket P \rrbracket?$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$P = x := x + 1$$

$$\text{In}(S) = (x = 1) \vee (x = 3)$$

$\tau(P) \stackrel{\text{def}}{=} \text{if } \text{In}(S) \text{ then}$
 if $\neg \text{In}(S)$ **then** $(y := 2; x := 0)$
 else $x := x + 1$ (@1)
 else $x := x + 1$ (@2)

$$\llbracket \tau(P) \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$P = x := x + 1$$

$$\text{In}(S) = (x = 1) \vee (x = 3)$$

$\tau(P) \stackrel{\text{def}}{=} \text{if } \text{In}(S) \text{ then}$
 if $\neg \text{In}(S)$ **then** $(y := 2; x := 0)$
 else $x := x + 1$ (@1)
 else $x := x + 1$ (@2)

$$\llbracket \tau(P) \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

$$\llbracket P \rrbracket^{\mathcal{A}} \alpha(S) < \llbracket \tau(P) \rrbracket^{\mathcal{A}} \alpha(S)?$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$P = x := x + 1$$

$$\text{In}(S) = (x = 1) \vee (x = 3)$$

$\tau(P) \stackrel{\text{def}}{=} \text{if } \text{In}(S) \text{ then}$
 if $\neg \text{In}(S)$ **then** $(y := 2; x := 0)$
 else $x := x + 1$ (@1)
 else $x := x + 1$ (@2)

$$\llbracket \tau(P) \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

$$\llbracket P \rrbracket^{\mathcal{A}} \alpha(S) < \llbracket \tau(P) \rrbracket^{\mathcal{A}} \alpha(S)? \quad \text{OK!}$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

1. variable finite인 경우

$$\text{var}((\gamma \circ \alpha)S) = \text{var}(S)$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

1. variable finite인 경우

$$\text{var}((\gamma \circ \alpha)S) = \text{var}(S)$$

2. Halting Problem의 도움으로,

$S \subset (\gamma \circ \alpha)S$ 인 s 가 존재

7 REDUCING COMPLETENESS TO INCOMPLETENESS

1. variable finite인 경우

$$\text{var}((\gamma \circ \alpha)S) = \text{var}(S)$$

2. Halting Problem의 도움으로,

$S \subset (\gamma \circ \alpha)S$ 인 s 가 존재

3. variable finiteness를 이용하여

$m' \notin (\gamma \circ \alpha)S$ 인 m' 이 존재

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$\text{var}((\gamma \circ \alpha)S) = \text{var}(S)$$

$$S \subset (\gamma \circ \alpha)S$$

$$m' \notin (\gamma \circ \alpha)S$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$\text{var}((\gamma \circ \alpha)S) = \text{var}(S)$$

$$S \subset (\gamma \circ \alpha)S$$

$$\mathfrak{m}' \notin (\gamma \circ \alpha)S$$

4. 다음과 같이 프로그램 변환

$$\tau_{\mathfrak{m}', S}(P) \stackrel{\text{def}}{=} \begin{array}{l} \text{if } \text{In}(S) \text{ then} \\ \quad \text{if } \neg \text{In}(S) \text{ then } \text{Set}(\mathfrak{m}', S) \\ \quad \text{else } P \\ \text{else } P \end{array}$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$\text{var}((\gamma \circ \alpha)S) = \text{var}(S)$$

$$S \subset (\gamma \circ \alpha)S$$

$$\mathfrak{m}' \notin (\gamma \circ \alpha)S$$

4. 다음과 같이 프로그램 변환

$$\tau_{\mathfrak{m}', S}(P) \stackrel{\text{def}}{=} \begin{array}{l} \text{if } \text{In}(S) \text{ then} \\ \quad \text{if } \neg \text{In}(S) \text{ then } \text{Set}(\mathfrak{m}', S) \\ \quad \text{else } P \\ \text{else } P \end{array}$$

$$\llbracket \tau(P) \rrbracket = \llbracket P \rrbracket?$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$\text{var}((\gamma \circ \alpha)S) = \text{var}(S)$$

$$S \subset (\gamma \circ \alpha)S$$

$$\mathfrak{m}' \notin (\gamma \circ \alpha)S$$

4. 다음과 같이 프로그램 변환

$$\tau_{\mathfrak{m}', S}(P) \stackrel{\text{def}}{=} \begin{array}{l} \text{if } \text{In}(S) \text{ then} \\ \quad \text{if } \neg \text{In}(S) \text{ then } \text{Set}(\mathfrak{m}', S) \\ \quad \text{else } P \\ \text{else } P \end{array}$$

$$\llbracket \tau(P) \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$\text{var}((\gamma \circ \alpha)S) = \text{var}(S)$$

$$S \subset (\gamma \circ \alpha)S$$

$$\mathfrak{m}' \notin (\gamma \circ \alpha)S$$

4. 다음과 같이 프로그램 변환

$$\tau_{\mathfrak{m}', S}(P) \stackrel{\text{def}}{=} \begin{array}{l} \text{if } \text{In}(S) \text{ then} \\ \quad \text{if } \neg \text{In}(S) \text{ then Set}(\mathfrak{m}', S) \\ \quad \text{else } P \\ \text{else } P \end{array}$$

$$\llbracket \tau(P) \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

$$\llbracket \tau(P) \rrbracket^{\mathcal{A}} \alpha(S) > \llbracket P \rrbracket^{\mathcal{A}} \alpha(S)?$$

7 REDUCING COMPLETENESS TO INCOMPLETENESS

$$\text{var}((\gamma \circ \alpha)S) = \text{var}(S)$$

$$S \subset (\gamma \circ \alpha)S$$

$$\mathfrak{m}' \notin (\gamma \circ \alpha)S$$

4. 다음과 같이 프로그램 변환

$$\tau_{\mathfrak{m}', S}(P) \stackrel{\text{def}}{=} \begin{array}{l} \text{if } \text{In}(S) \text{ then} \\ \quad \text{if } \neg \text{In}(S) \text{ then } \text{Set}(\mathfrak{m}', S) \\ \quad \text{else } P \\ \text{else } P \end{array}$$

$$\llbracket \tau(P) \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

$$\llbracket \tau(P) \rrbracket^{\mathcal{A}} \alpha(S) > \llbracket P \rrbracket^{\mathcal{A}} \alpha(S)? \quad \text{OK!}$$

8 RICE EXTENSIONALITY OF THE ABSTRACT SEMANTICS

variable finite가 아닌 경우

$$\text{var}(S) \subset \text{var}((\gamma \circ \alpha)S)$$

8 RICE EXTENSIONALITY OF THE ABSTRACT SEMANTICS

variable finite가 아닌 경우

$$\text{var}(S) \subset \text{var}((\gamma \circ \alpha)S)$$

x 와 m 을 잘 잡아서

$$x \in \text{var}((\gamma \circ \alpha)S) \setminus \text{var}(S) \text{ with } m(x) = v \neq \$$$

$$P \stackrel{\text{def}}{=} \text{while tt do skip}$$

$$Q \stackrel{\text{def}}{=} \begin{array}{l} \text{if } x \neq v \text{ then} \\ \quad \text{if } x = v \text{ then skip} \\ \quad \text{else } P \\ \text{else } P \end{array}$$

8 RICE EXTENSIONALITY OF THE ABSTRACT SEMANTICS

$$\text{var}(S) \subset \text{var}((\gamma \circ \alpha)S)$$

$$x \in \text{var}((\gamma \circ \alpha)S) \setminus \text{var}(S) \text{ with } \mathfrak{m}(x) = v \neq \$$$

$$P \stackrel{\text{def}}{=} \text{while tt do skip}$$

$$Q \stackrel{\text{def}}{=} \begin{array}{l} \text{if } x \neq v \text{ then} \\ \quad \text{if } x = v \text{ then skip} \\ \quad \text{else } P \\ \text{else } P \end{array}$$

8 RICE EXTENSIONALITY OF THE ABSTRACT SEMANTICS

$$\text{var}(S) \subset \text{var}((\gamma \circ \alpha)S)$$

$$x \in \text{var}((\gamma \circ \alpha)S) \setminus \text{var}(S) \text{ with } \mathfrak{m}(x) = v \neq \$$$

$$P \stackrel{\text{def}}{=} \text{while tt do skip}$$

$$Q \stackrel{\text{def}}{=} \begin{array}{l} \text{if } x \neq v \text{ then} \\ \quad \text{if } x = v \text{ then skip} \\ \quad \text{else } P \\ \text{else } P \end{array}$$

$$\llbracket P \rrbracket = \llbracket Q \rrbracket = \perp$$

8 RICE EXTENSIONALITY OF THE ABSTRACT SEMANTICS

$$\text{var}(S) \subset \text{var}((\gamma \circ \alpha)S)$$

$$x \in \text{var}((\gamma \circ \alpha)S) \setminus \text{var}(S) \text{ with } \mathfrak{m}(x) = v \neq \$$$

$$P \stackrel{\text{def}}{=} \text{while tt do skip}$$

$$Q \stackrel{\text{def}}{=} \begin{array}{l} \text{if } x \neq v \text{ then} \\ \quad \text{if } x = v \text{ then skip} \\ \quad \text{else } P \\ \text{else } P \end{array}$$

$$\llbracket P \rrbracket = \llbracket Q \rrbracket = \perp$$

$$\llbracket P \rrbracket^{\mathcal{A}} \alpha(S) < \llbracket Q \rrbracket^{\mathcal{A}} \alpha(S)$$

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

(부분)역함수 관계에 있는 두 함수 g, h 를 이용

$P_{x,g}; P_{x,h}; P$

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

(부분)역함수 관계에 있는 두 함수 g, h 를 이용

$P_{x,g}; P_{x,h}; P$

$$g(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k < 2, \\ k + 1 & \text{if } k \geq 2. \end{cases}$$

$P_{x,g} =$ **if** $(x \geq 2)$ **then** $x := x + 1$
else skip

$$h(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k \leq 1, \\ -10 & \text{if } k = 2, \\ k - 1 & \text{if } k > 2. \end{cases}$$

$P_{x,h} =$ **if** $(x = 2)$ **then** $x := -10$
else if $(x > 2)$ **then** $x := x - 1$
else skip

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

$$g(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k < 2, \\ k + 1 & \text{if } k \geq 2. \end{cases}$$

$$P_{x,g} = \text{if } (x \geq 2) \text{ then } x := x + 1 \\ \text{else skip}$$

$$h(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k \leq 1, \\ -10 & \text{if } k = 2, \\ k - 1 & \text{if } k > 2. \end{cases}$$

$$P_{x,h} = \text{if } (x = 2) \text{ then } x := -10 \\ \text{else if } (x > 2) \text{ then } x := x - 1 \\ \text{else skip}$$

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

$$g(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k < 2, \\ k + 1 & \text{if } k \geq 2. \end{cases}$$

$$P_{x,g} = \text{if } (x \geq 2) \text{ then } x := x + 1 \\ \text{else skip}$$

$$h(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k \leq 1, \\ -10 & \text{if } k = 2, \\ k - 1 & \text{if } k > 2. \end{cases}$$

$$P_{x,h} = \text{if } (x = 2) \text{ then } x := -10 \\ \text{else if } (x > 2) \text{ then } x := x - 1 \\ \text{else skip}$$

$$P = x := x + 1$$

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

$$g(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k < 2, \\ k + 1 & \text{if } k \geq 2. \end{cases}$$

$$P_{x,g} = \text{if } (x \geq 2) \text{ then } x := x + 1 \\ \text{else skip}$$

$$h(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k \leq 1, \\ -10 & \text{if } k = 2, \\ k - 1 & \text{if } k > 2. \end{cases}$$

$$P_{x,h} = \text{if } (x = 2) \text{ then } x := -10 \\ \text{else if } (x > 2) \text{ then } x := x - 1 \\ \text{else skip}$$

$$P = x := x + 1$$

$S = \mathfrak{m}[x \mapsto \mathbb{N}]$ 라고 두면,

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

$$g(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k < 2, \\ k + 1 & \text{if } k \geq 2. \end{cases}$$

$$P_{x,g} = \text{if } (x \geq 2) \text{ then } x := x + 1 \\ \text{else skip}$$

$$h(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k \leq 1, \\ -10 & \text{if } k = 2, \\ k - 1 & \text{if } k > 2. \end{cases}$$

$$P_{x,h} = \text{if } (x = 2) \text{ then } x := -10 \\ \text{else if } (x > 2) \text{ then } x := x - 1 \\ \text{else skip}$$

$$P = x := x + 1$$

$S = \mathfrak{m}[x \mapsto \mathbb{N}]$ 라고 두면,

$$\llbracket P_{x,g}; P_{x,h}; P \rrbracket = \llbracket P \rrbracket?$$

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

$$g(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k < 2, \\ k + 1 & \text{if } k \geq 2. \end{cases}$$

$$P_{x,g} = \text{if } (x \geq 2) \text{ then } x := x + 1 \\ \text{else skip}$$

$$h(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k \leq 1, \\ -10 & \text{if } k = 2, \\ k - 1 & \text{if } k > 2. \end{cases}$$

$$P_{x,h} = \text{if } (x = 2) \text{ then } x := -10 \\ \text{else if } (x > 2) \text{ then } x := x - 1 \\ \text{else skip}$$

$$P = x := x + 1$$

$S = \mathfrak{m}[x \mapsto \mathbb{N}]$ 라고 두면,

$$\llbracket P_{x,g}; P_{x,h}; P \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

$$g(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k < 2, \\ k + 1 & \text{if } k \geq 2. \end{cases}$$

$$P_{x,g} = \text{if } (x \geq 2) \text{ then } x := x + 1 \\ \text{else skip}$$

$$h(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k \leq 1, \\ -10 & \text{if } k = 2, \\ k - 1 & \text{if } k > 2. \end{cases}$$

$$P_{x,h} = \text{if } (x = 2) \text{ then } x := -10 \\ \text{else if } (x > 2) \text{ then } x := x - 1 \\ \text{else skip}$$

$$P = x := x + 1$$

$S = \mathfrak{m}[x \mapsto \mathbb{N}]$ 라고 두면,

$$\llbracket P_{x,g}; P_{x,h}; P \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

$$\llbracket P_{x,g}; P_{x,h}; P \rrbracket^{\mathcal{A}} \alpha(S) > \llbracket P \rrbracket^{\mathcal{A}} \alpha(S)?$$

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

$$g(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k < 2, \\ k + 1 & \text{if } k \geq 2. \end{cases}$$

$$P_{x,g} = \text{if } (x \geq 2) \text{ then } x := x + 1 \\ \text{else skip}$$

$$h(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k \leq 1, \\ -10 & \text{if } k = 2, \\ k - 1 & \text{if } k > 2. \end{cases}$$

$$P_{x,h} = \text{if } (x = 2) \text{ then } x := -10 \\ \text{else if } (x > 2) \text{ then } x := x - 1 \\ \text{else skip}$$

$$P = x := x + 1$$

$S = \mathfrak{m}[x \mapsto \mathbb{N}]$ 라고 두면,

$$\llbracket P_{x,g}; P_{x,h}; P \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

$$\llbracket P_{x,g}; P_{x,h}; P \rrbracket^{\mathcal{A}} \alpha(S) > \llbracket P \rrbracket^{\mathcal{A}} \alpha(S)? \quad \text{OK!}$$

9 MAKING ABSTRACTIONS INCOMPLETE BY DATA-FLOW TRANSFORMATIONS

$$g(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k < 2, \\ k + 1 & \text{if } k \geq 2. \end{cases}$$

$$P_{x,g} = \text{if } (x \geq 2) \text{ then } x := x + 1 \\ \text{else skip}$$

$$h(k) \stackrel{\text{def}}{=} \begin{cases} k & \text{if } k \leq 1, \\ -10 & \text{if } k = 2, \\ k - 1 & \text{if } k > 2. \end{cases}$$

$$P_{x,h} = \text{if } (x = 2) \text{ then } x := -10 \\ \text{else if } (x > 2) \text{ then } x := x - 1 \\ \text{else skip}$$

$$P = x := x + 1$$

$S = \mathfrak{m}[x \mapsto \mathbb{N}]$ 라고 두면,

$$\llbracket P_{x,g}; P_{x,h}; P \rrbracket = \llbracket P \rrbracket? \quad \text{OK!}$$

$$\llbracket P_{x,g}; P_{x,h}; P \rrbracket^{\mathcal{A}} \alpha(S) > \llbracket P \rrbracket^{\mathcal{A}} \alpha(S)? \quad \text{OK!}$$

단, 항상 되는 건 아님!

결론

결론

- variable finiteness 상관 없이, 정적 분석 골탕먹이기가 항상 가능

결론

- variable finiteness 상관 없이, 정적 분석 골탕먹이기가 항상 가능
- 정적 분석을 골탕먹이는 프로그램만으로도, 튜링 완전한 집합 형성

한계점 및 발전 방향

한계점

계속 “존재성”만 얘기하고, 증명하고 있음 구체적으로 어떻게
프로그램을 변환하거나 찾는지에 대한 내용이 전혀 없음

한계점

계속 “존재성”만 얘기하고, 증명하고 있음 구체적으로 어떻게 프로그램을 변환하거나 찾는지에 대한 내용이 전혀 없음

- S 를 어떻게 찾을 것인가?

한계점

계속 “존재성”만 얘기하고, 증명하고 있음 구체적으로 어떻게 프로그램을 변환하거나 찾는지에 대한 내용이 전혀 없음

- S 를 어떻게 찾을 것인가?
- $\ln(S)$ 를 어떻게 프로그램으로 구현할 것인가?

한계점

계속 “존재성”만 얘기하고, 증명하고 있음 구체적으로 어떻게 프로그램을 변환하거나 찾는지에 대한 내용이 전혀 없음

- S 를 어떻게 찾을 것인가?
- $\ln(S)$ 를 어떻게 프로그램으로 구현할 것인가?

0부터 합성하는 것이 아니라, 기존에 있던 프로그램을 변환시키는 방법을 사용할 것이라면 이 방법을 찾는 것이 목표

한계점

계속 “존재성”만 얘기하고, 증명하고 있음 구체적으로 어떻게 프로그램을 변환하거나 찾는지에 대한 내용이 전혀 없음

- S 를 어떻게 찾을 것인가?
- $\ln(S)$ 를 어떻게 프로그램으로 구현할 것인가?

0부터 합성하는 것이 아니라, 기존에 있던 프로그램을 변환시키는 방법을 사용할 것이라면 이 방법을 찾는 것이 목표

혹은, 기계적으로는 불가능하다는 것을 증명하는 것이 목표

발전 방향

- 골탕 합성 연구의 기반은 Rice theorem에서 나온 존재성

발전 방향

- 골탕 합성 연구의 기반은 Rice theorem에서 나온 존재성
- 그러나 여전히 합성이 기계적으로 가능하다는 보장을 가지고 출발한 것은 아님

발전 방향

- 골탕 합성 연구의 기반은 Rice theorem에서 나온 존재성
- 그러나 여전히 합성이 기계적으로 가능하다는 보장을 가지고 출발한 것은 아님
- 이 논문의 아이디어를 응용, if문으로 구성된 기본 틀을 주고, 그 안을 합성하게 하는 방법을 생각해 보자

```

if            then
  if  $\neg$             then 
  else  $P$ 
else  $P$ 
    
```


발전 방향

- 골탕 합성 연구의 기반은 Rice theorem에서 나온 존재성
- 그러나 여전히 합성이 기계적으로 가능하다는 보장을 가지고 출발한 것은 아님
- 이 논문의 아이디어를 응용, if문으로 구성된 기본 틀을 주고, 그 안을 합성하게 하는 방법을 생각해 보자
- +, Rice theorem을 직접적으로 활용하기 위해 while문을 기본 틀로 주는 것도 가능

if [] then
 if $\neg$ [] then
 else P
 else P

while do

발전 방향

- “실제로는 top이 아니다” 를 어떻게 구성하면서 합성해나갈지는 여전히 남아 있는 문제
- Giacobazzi의 다른 난독화 관련 논문들도 뒤져보기
- 가짜 경보가 나는 (이미 있는)큰 프로그램을 깎아서 작은 조각을 만드는 방법을 쓸 때, 변환 아이디어를 적용할 수도

감사합니다